系统 5: 文件读写监控

从轮询到事件通知

A 问题是什么

编辑器和同步工具怎样发现文件已经变化，而不必让用户手动刷新？本项目用 C++ 观察一个测试目录。Basic 周期性扫描文件大小和修改时间，比较前后快照；Advance 在 Linux 上使用 inotify 接收访问、修改和目录条目变化通知，理解轮询与事件驱动的取舍。监控器读取目录信息并打印日志，测试文件的创建和修改由另一个程序或编辑器完成。

阶段	工作量	难度	主要成果
Basic	一周约 10-20 小时	medium	非递归的目录轮询监控器
Advance	Basic 后追加六周	medium；可选阅读含 hard	使用 inotify、过滤及事件聚合的监控器

需要 C++17 编译器。Basic 使用标准库，可在支持 `std::filesystem` 的环境运行；Advance 需要 Linux，可使用已有的 Linux 机器或 WSL 环境。easy 为单一概念或规则，medium 需要处理 API、状态与错误，hard 需要理解并发或复杂系统机制。估时包含基础环境检查，但不包含从零安装整套操作系统。

Basic：比较目录快照

1 需要回答的问题

编号	问题	回答要求
Q1	监控程序怎样知道目录发生了变化?	说明一次快照保存什么、前后两次如何比较
Q2	文件大小与最后修改时间分别表示什么?	说明为什么它们不是文件内容本身，也不是精确的操作次数
Q3	怎样区分新增、删除和修改?	用前后快照中的路径是否存在、元数据是否变化来解释
Q4	扫描间隔越短一定越好吗?	比较检测延迟与扫描开销；说明扫描之间创建又删除的文件可能被漏掉
Q5	旧快照为 `a:(10,1)`, `b:(20,1)`，新快照为 `a:(12,2)`, `c:(20,1)`，应输出什么?	括号为大小与修改时间；写出 MODIFY、DELETE、CREATE，并说明单凭这些信息不能认定 b 被重命名为 c

2 代码任务

用 `map<string, FileInfo>` 保存“相对文件名 → 大小、最后修改时间”。实现一个扫描函数和一个比较函数，时间可以保留为 `filesystem::file_time_type`，不要求转换为日历时间。

运行形式可自行设计，例如 `./monitor` 测试目录轮询次数。启动时先保存基准快照，随后每隔约一秒扫描一次，共执行指定次数后退出。仅处理指定目录下的普通文件，不递归，不跟随符号链接；子目录跳过。

比较结果	输出
新快照有、旧快照没有	`CREATE` 文件名
旧快照有、新快照没有	`DELETE` 文件名
两边都有，大小或修改时间不同	`MODIFY` 文件名
两边都有且这两项相同	不输出

每轮日志按文件名字典序输出，便于复现。初始目录内容只作为基线，不记为新增。目录不存在、无法读取或某次扫描失败时显示错误；不能把失败的扫描当作空目录，产生一批虚假的 DELETE。扫描成功后才能更新基线。

这个阶段只能发现采样时看到的元数据差异，不能监控纯读取，也不能保证发现所有内容修改。重命名按“旧路径删除 + 新路径创建”处理。监控器本身不创建、删除或修改被监控文件，也不要求多线程。

3 演示与验收

在一个专用测试目录中，启动监控后分别创建、追加、删除文件，每步等待至少两个扫描周期，检查相应日志。另测已有文件作为基线、空目录、子目录被忽略、非法路径、内容不变的重复扫描。

对比较函数再使用 Q5 这种人工快照测试；这样不依赖编辑器保存方式或真实时间精度。提交源码、运行环境及命令、测试记录和 Q1-Q5 回答。Basic 不要求 presentation，不要求用 OJ 验证依赖本机文件事件的行为。

4 参考资料

先读 B1、B2 的指定部分；B3 补充路径和映射，B4 留作事件通知的预览。

编号	资料与语言	建议范围
B1	Microsoft Learn：文件系统导航，中文	路径、遍历目录与文件；示例按 C++17 理解
B2	Microsoft Learn：filesystem，中文	查 `directory_iterator`、`is_regular_file`、`file_size`、`last_write_time` 和错误处理接口
B3	Hello 算法：哈希表，中文	保存和比较文件快照中的键值映射
B4	inotify(7)，英文 Linux 手册	只读开头的用途介绍，暂不读完整 API 与限制

Advance：Linux 事件驱动监控

1 Goals 与难度

G1-G7 均需完成，G6 选择一个阅读方向。必做程序仍只监控一个目录的直接子项。

Goal	完成要求	难度
G1	解释系统调用、轮询与事件通知的区别，说明 inotify 通知来自哪里	easy
G2	创建 inotify 实例、添加 watch，读取并正确遍历事件缓冲区	medium
G3	识别访问、修改、创建、删除及移动事件，区分文件与目录	medium
G4	加入简单过滤和 MODIFY 日志聚合，解释收到的事件数为何不等于真实写操作数	medium
G5	实现有限等待、退出清理和队列溢出提示，避免空闲时无法退出	medium
G6	选择一个系统方向阅读，不要求实现对应扩展	见方向表
G7	对比两种监控方式，整理测试、阅读笔记和 presentation	medium

2 实现要求与范围

直接调用 Linux inotify API，核心接收过程自行编写。建议单线程使用 `inotify_init1(IN_NONBLOCK | IN_CLOEXEC)`，配合带有限超时的 `poll`；不以 `system("inotifywait")` 或监控库替代。程序只接收文件事件，不读取文件内容来推断事件。

- 用 `inotify_add_watch` 监听指定目录；记录 watch descriptor 与目录路径的关系。
- 关注 `IN_ACCESS`、`IN_MODIFY`、`IN_CREATE`、`IN_DELETE`、`IN_MOVED_FROM`、`IN_MOVED_TO`；同时订阅 `IN_DELETE_SELF`、`IN_MOVE_SELF` 以发现监控目录自身失效。用位运算判断事件掩码，并识别 `IN_ISDIR`、`IN_IGNORED`。同一个事件可能带多个标志。
- 一次 `read` 可能包含多条变长记录。每条占 `sizeof(inotify_event) + len` 字节；先检查剩余长度，注意缓冲区对齐、`len == 0` 和文件名终止符。不要按固定步长处理全部事件。
- Basic 的 CREATE/MODIFY/DELETE 文本格式可以保留；读取通知输出 ACCESS。移动事件先分别输出 MOVED_FROM/MOVED_TO 与 cookie，能够说明同一 cookie 用于关联。跨监控边界可能只收到一半，不要求完整还原重命名。

- 过滤只需一种简单规则，例如只显示 `.txt` 文件，或忽略名称以 `.` 开头的条目。规则作用于日志，不影响必要的状态维护。

聚合约定：对通过过滤的同一路径 MODIFY 事件，保存首次、末次接收时刻和收到的事件条数；连续一秒没有新 MODIFY 后输出一条汇总并清除记录。采用单调时钟检查静默时间，退出时输出尚未完成的汇总。这是“按静默间隔聚合”；持续不断修改时可能一直等待到停止或退出。计数只能称为“收到的通知条数”，不能称为文件真实写了多少次。

退出与错误：定期检查退出条件，空闲和高频事件下都应能退出。若使用 SIGINT，处理函数只设置 `volatile std::sig_atomic_t` 标志，资源释放、输出和容器操作在主循环完成；不要在信号处理函数中操作队列。正确区分 `EAGAIN`、`EINTR` 与真正错误，退出时移除 watch 并关闭描述符。

遇到 `IN_Q_OVERFLOW` 要输出“事件可能丢失”，不能继续宣称日志完整；监控目录被删除、移走或 watch 失效时应提示并结束本次监控。可利用 Basic 快照检查最终目录状态，但快照无法补回全部历史操作。inotify 也不提供每次事件对应的进程身份、读写字节数，不能将本项目当作完整文件审计系统。inotify(7) 的限制说明

3 测试与比较

在同一测试目录对轮询和事件版分别执行创建、读取、追加、删除、移动与连续修改。记录检测到的类型、重复通知和可观察延迟；计时方法与扫描间隔须说明，不设“必定小于某毫秒”的跨机器验收条件。

重点测试：一个缓冲区有多条事件、无事件时退出、高频事件时退出、过滤生效、聚合超时与退出冲刷、监控目录失效。溢出难以稳定触发时，可将解析和状态处理拆成函数，用人工事件验证错误分支。不要要求每次编辑器保存只产生一条 MODIFY。

4 参考资料与可选方向

编号	资料与语言	建议范围
A1	rCore：操作系统的系统调用接口，中文	用户程序如何请求内核服务，支持 G1
A2	Microsoft Learn：filesystem，中文	复习路径、目录枚举和错误处理，支持对照实验
A3	inotify(7)，英文	API、事件掩码、变长结构及 Limitations，是 G2-G5 的主要接口资料
A4	poll(2)，英文	就绪通知、超时与返回值，支持可退出的事件循环
A5	rCore：并发中的问题，中文	互斥、同步与死锁，供并发方向阅读
A6	signal-safety(7)，英文	理解信号处理函数为什么应尽量简单，不要求背函数列表

先读 A1，再查 A3 的核心 API 与示例，其余按需阅读。无需通读全部手册或内核源码。

G6 可选方向	难度	需要回答的问题
事件通知的局限（A3）	medium	为什么事件会合并或丢失？为什么最终状态正确不代表历史日志完整？
递归监控（A2、A3）	medium	新子目录出现时怎样补 watch？目录移动为什么让路径映射变复杂？
生产者-消费者（A5）	hard	接收事件与处理日志分成两个线程后，怎样协调队列、唤醒、容量限制与退出？

最终提交源码、编译运行说明、过滤与聚合规则、对比记录、一个方向的简短笔记和 presentation。

B 工作量安排与 Checkpoint

Basic：一周

内容	预计用时
环境检查与目录 API 学习	3-5 小时
快照与比较函数	3-4 小时
加入轮询并手工验证	3-4 小时
错误处理、Q1-Q5 与说明	2-4 小时
合计	11-17 小时，预留至 20 小时

Checkpoint：人工快照比较正确，真实目录操作能产生对应日志，并能说明漏报可能性。

Advance：追加六周

时间	对应 **Goal**	**Checkpoint**
第 1-2 周	G1：easy；G2、G3：medium	单目录接收通知，正确解析多事件与事件标志
第 3-4 周	G4、G5：medium	过滤与聚合符合约定，空闲时也能退出并清理资源
第 5-6 周	G6：按方向；G7：medium	完成两种方式的对照测试、阅读与 presentation

C 问题的来龙去脉与发展过程

最直接的监控办法是反复询问文件当前状态。文件增多或扫描间隔缩短后，重复扫描开销随之增加；采样之间发生又消失的变化还可能无法被看见。事件通知把“主动扫描全部状态”转为“状态变化时接收通知”，但需要额外处理队列、错误和失效情况。

Linux 曾使用 dnotify，后来引入 inotify；inotify 自 Linux 2.6.13 起提供文件系统事件通知。它改善了通知接口，却仍有非递归、事件合并、队列溢出等边界。项目的重点是理解为何改用事件驱动，以及这种改进为何仍不能代替完整历史记录。Linux inotify 手册